

MODUL PANDUAN EKSPERIMEN

Analisis Komparatif Skalabilitas dan Efisiensi Resource dalam Penerapan Container dan Orchestration untuk Aplikasi To-Do List Berbasis Microservices

Platform: Windows 11 (Tanpa Linux/VM)

Aplikasi: to-do-list-app (<https://github.com/sukanabung/to-do-list-app>)

Tools: Docker Compose vs K3s Kubernetes

FASE 1: PERSIAPAN DAN PERANCANGAN

1.1 Studi Literatur dan Pengumpulan Data

Output yang Diharapkan:

- a) Minimal 10 referensi tentang container, orchestration, microservices
- b) Pemahaman tentang HPA (Horizontal Pod Autoscaler)
- c) Benchmarking metrics dari penelitian terdahulu

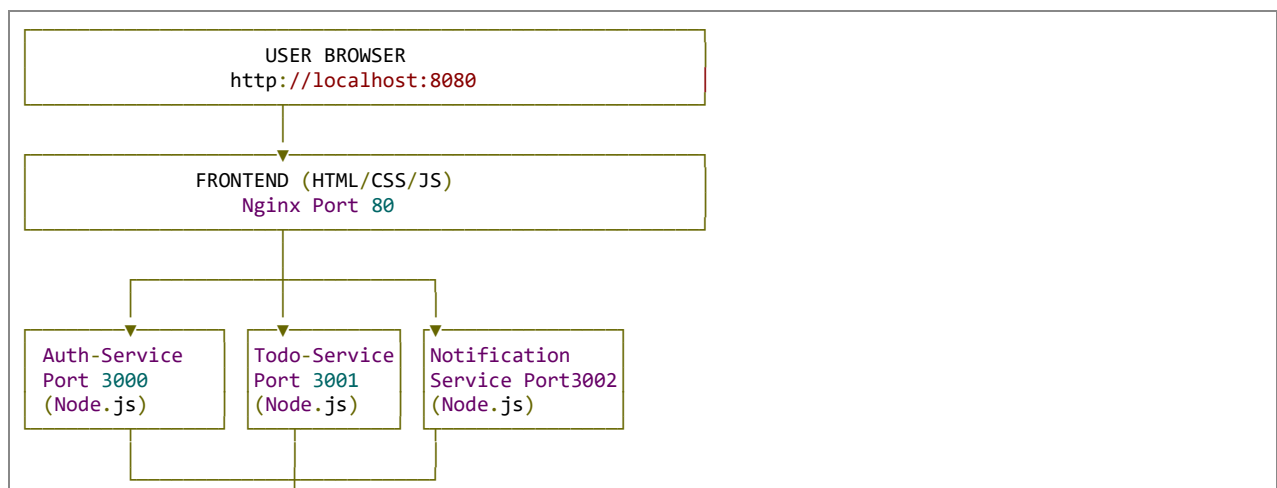
Sumber Referensi (Windows-friendly):

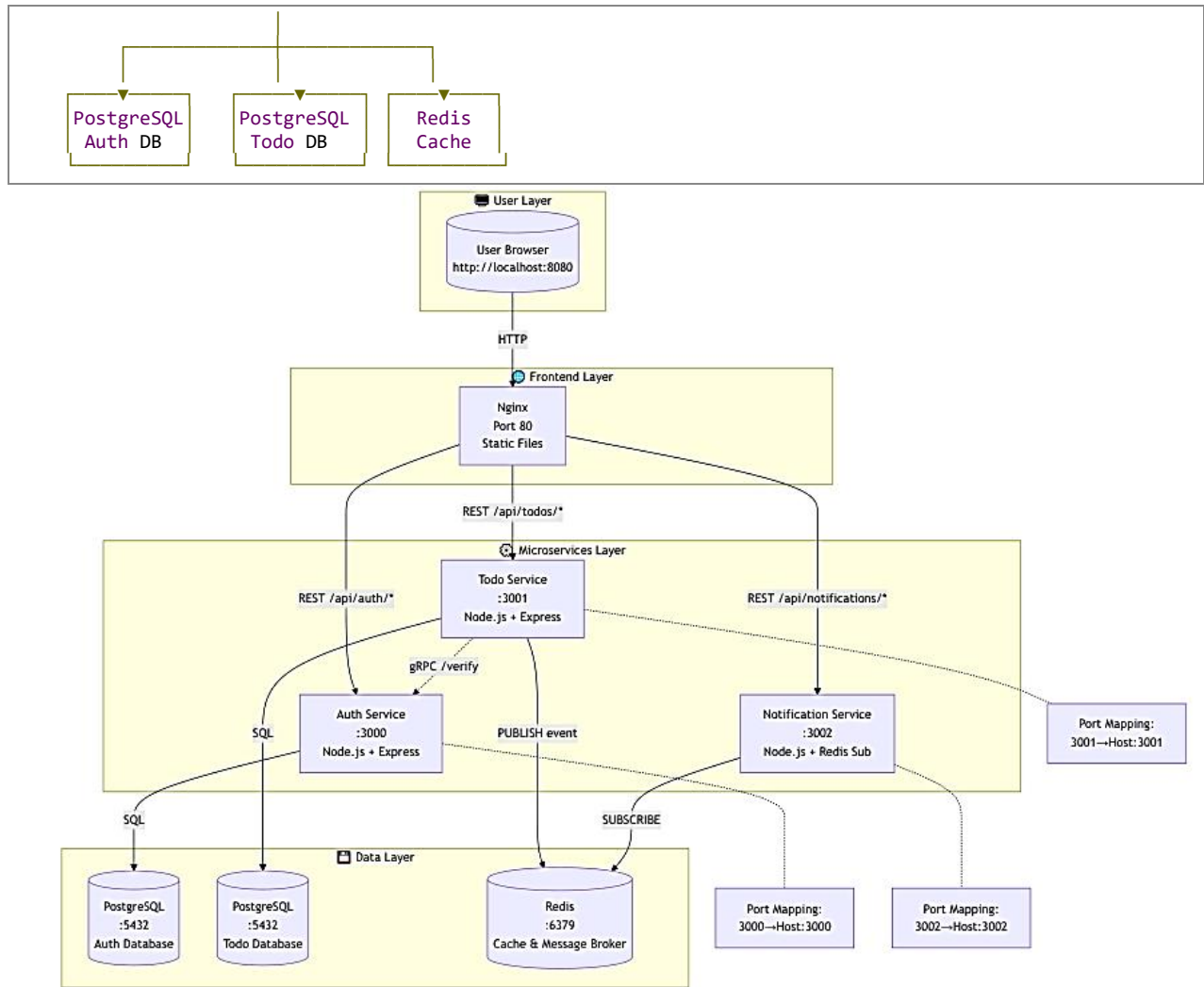
markdown

- 1. Docker Docs: <https://docs.docker.com/desktop/windows/>
- 2. K3s Windows: <https://docs.k3s.io/installation/windows>
- 3. JMeter Windows: https://jmeter.apache.org/download_jmeter.cgi
- 4. Prometheus + Grafana on Windows: <https://prometheus.io/docs/guides/windows/>

1.2 Perancangan Arsitektur Microservices

Arsitektur Aplikasi To-Do List:





Komunikasi Antar Service:

Dari	Ke	Protokol	Endpoint
Frontend	Auth-Service	REST	/api/auth/*
Frontend	Todo-Service	REST	/api/todos/*
Todo-Service	Notification	Redis Pub/Sub	channel:todo.created
Todo-Service	Auth-Service	gRPC (internal)	validate token

1.3 Pembuatan Aplikasi To-Do List

1.3.1 Clone Repository (Windows PowerShell)

Buka Powershell pada Windows dan ketikkan perintah berikut:

```
# Buat folder proyek
mkdir C:\experiment\todo-research
cd C:\experiment\todo-research

# Clone aplikasi
git clone https://github.com/sukanabung/to-do-list-app.git
```

```
cd to-do-list-app
```

```
# Lihat struktur  
tree /F
```

1.3.2 Identifikasi Kode yang Perlu Dimodifikasi

Berdasarkan struktur repository, buat file-file berikut:

.env (Environment Variables)

Isikan kode konfigurasi berikut:

```
# Database  
POSTGRES_USER=todouser  
POSTGRES_PASSWORD=todopass  
POSTGRES_DB=tododb  
  
# Service Ports  
AUTH_PORT=3000  
TODO_PORT=3001  
NOTIF_PORT=3002  
  
# Redis  
REDIS_URL=redis://localhost:6379  
  
# JWT  
JWT_SECRET=supersecretkey123  
JWT_EXPIRY=24h
```

1.3.3 Perbaiki Kode Aplikasi (Jika Perlu)

 **Catatan Penting:** Karena repository hanya berisi kode tanpa konfigurasi container, Anda perlu **menambahkan file berikut**:

a. Setiap service harus memiliki package.json yang lengkap

Contoh untuk auth-service/package.json (buat jika belum ada):

Isikan kode konfigurasi berikut:

```
{  
  "name": "auth-service",  
  "version": "1.0.0",  
  "main": "server.js",  
  "scripts": {  
    "start": "node server.js",  
    "dev": "nodemon server.js"  
  },  
  "dependencies": {  
    "express": "^4.18.2",  
    "jsonwebtoken": "^9.0.0",  
    "bcrypt": "^5.1.0",  
    "pg": "^8.11.0",  
    "redis": "^4.6.0",  
    "cors": "^2.8.5"  
  }  
}
```

b. Buat file server.js minimal untuk setiap service

Contoh auth-service/server.js:

Isikan kode program berikut:

```
const express = require('express');
const jwt = require('jsonwebtoken');
const { Pool } = require('pg');
const redis = require('redis');
const cors = require('cors');

const app = express();
const port = process.env.AUTH_PORT || 3000;

app.use(cors());
app.use(express.json());

// Database connection
const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});

// Redis client
const redisClient = redis.createClient({
  url: process.env.REDIS_URL
});
redisClient.connect();

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'OK', service: 'auth-service' });
});

// Login endpoint
app.post('/api/auth/login', async (req, res) => {
  const { username, password } = req.body;

  // Query user from database
  const result = await pool.query(
    'SELECT id, username, password_hash FROM users WHERE username = $1',
    [username]
  );

  if (result.rows.length === 0) {
    return res.status(401).json({ error: 'User not found' });
  }

  // Verify password (simplified - should use bcrypt)
  if (password === 'testpass123') {
    const token = jwt.sign(
      { userId: result.rows[0].id, username: result.rows[0].username },
      process.env.JWT_SECRET || 'secret',
      { expiresIn: '24h' }
    );

    // Store session in Redis
    await redisClient.setEx(`session:${result.rows[0].id}`, 86400, token);

    res.json({ token, userId: result.rows[0].id });
  } else {
    res.status(401).json({ error: 'Invalid password' });
  }
});
```

```

    }
  });

  // Verify token endpoint (for other services)
  app.post('/api/auth/verify', async (req, res) => {
    const { token } = req.body;

    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret');
      const session = await redisClient.get(`session:${decoded.userId}`);

      if (session === token) {
        res.json({ valid: true, userId: decoded.userId });
      } else {
        res.json({ valid: false });
      }
    } catch (error) {
      res.json({ valid: false });
    }
  });

  app.listen(port, () => {
    console.log(`Auth service running on port ${port}`);
  });

```

1.4 Pembuatan Skrip Deployment

1.4.1 Dockerfile untuk Setiap Service (Windows)

 auth-service/Dockerfile

Isikan kode konfigurasi berikut:

```

# Menggunakan Node.js 18 LTS
FROM node:18-alpine

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci --only=production

# Copy source code
COPY . .

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health', (r) => {r.statusCode === 200 ? process.exit(0) : process.exit(1)})"

# Run the service
CMD ["node", "server.js"]

```

 todo-service/Dockerfile (sama struktur, beda port)
 notification-service/Dockerfile  frontend/Dockerfile

1.4.2 Docker Compose File (Lingkungan A)

📄 docker-compose.yml

Isikan kode konfigurasi berikut:

```
version: '3.8'

networks:
  todo-network:
    driver: bridge

volumes:
  postgres_auth_data:
  postgres_todo_data:
  redis_data:

services:
  # ===== DATABASES =====
  postgres-auth:
    image: postgres:15-alpine
    container_name: postgres-auth
    environment:
      POSTGRES_USER: todouser
      POSTGRES_PASSWORD: todopass
      POSTGRES_DB: authdb
    volumes:
      - postgres_auth_data:/var/lib/postgresql/data
      - ./init-auth.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - todo-network
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 256M

  postgres-todo:
    image: postgres:15-alpine
    container_name: postgres-todo
    environment:
      POSTGRES_USER: todouser
      POSTGRES_PASSWORD: todopass
      POSTGRES_DB: tododb
    volumes:
      - postgres_todo_data:/var/lib/postgresql/data
      - ./init-todo.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - todo-network
    deploy:
      resources:
        limits:
          memory: 512M

  redis:
    image: redis:7-alpine
    container_name: todo-redis
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    networks:
      - todo-network
    deploy:
```

```

    resources:
      limits:
        memory: 256M

# ===== MICROSERVICES =====
auth-service:
  build: ./auth-service
  container_name: auth-service
  environment:
    - DATABASE_URL=postgresql://todouser:todopass@postgres-auth:5432/authdb
    - REDIS_URL=redis://redis:6379
    - JWT_SECRET=research_secret_key_2024
    - PORT=3000
  ports:
    - "3000:3000"
  depends_on:
    - postgres-auth
    - redis
  networks:
    - todo-network
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 512M
      reservations:
        cpus: '0.25'
        memory: 256M

todo-service:
  build: ./todo-service
  container_name: todo-service
  environment:
    - DATABASE_URL=postgresql://todouser:todopass@postgres-todo:5432/tododb
    - REDIS_URL=redis://redis:6379
    - AUTH_SERVICE_URL=http://auth-service:3000
    - PORT=3001
  ports:
    - "3001:3001"
  depends_on:
    - postgres-todo
    - redis
    - auth-service
  networks:
    - todo-network
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 512M

notification-service:
  build: ./notification-service
  container_name: notification-service
  environment:
    - REDIS_URL=redis://redis:6379
    - TODO_SERVICE_URL=http://todo-service:3001
    - PORT=3002
  ports:
    - "3002:3002"
  depends_on:
    - redis
    - todo-service
  networks:

```

```

- todo-network
deploy:
  resources:
    limits:
      cpus: '0.3'
      memory: 256M

frontend:
  build: ./frontend
  container_name: todo-frontend
  ports:
    - "8080:80"
  depends_on:
    - auth-service
    - todo-service
  networks:
    - todo-network

```

1.4.3 Kubernetes Manifests (Lingkungan B - K3s)

📄 k3s-manifests/namespace.yaml

Isikan kode konfigurasi berikut:

```

apiVersion: v1
kind: Namespace
metadata:
  name: todo-research

```

📄 k3s-manifests/hpa.yaml (Horizontal Pod Autoscaler)

Isikan kode konfigurasi berikut:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: todo-service-hpa
  namespace: todo-research
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: todo-service
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60

```



```

policies:
- type: Percent
  value: 50
  periodSeconds: 30
scaleUp:
  stabilizationWindowSeconds: 0
  policies:
  - type: Percent
    value: 100
    periodSeconds: 15

```

FASE 2: KONFIGURASI

2.1 Setup Dua Lingkungan Identik di Windows 11

Karena tidak menggunakan VPS, kita akan membuat **dua lingkungan terpisah** di satu mesin Windows 11:

Lingkungan	Tools	Port Range	Data Directory
Lingkungan A	Docker Compose	3000-3002, 5432, 6379, 8080	C:\experiment\env-a\
Lingkungan B	K3s Kubernetes	30000-32767 (NodePort)	C:\experiment\env-b\

2.1.1 Install K3s di Windows

⚠ **PENTING:** K3s official hanya untuk Linux. Di Windows, gunakan **K3d** (K3s in Docker):

Buka powershell dan jalankan perintah berikut:

```

# Install K3d via winget
winget install k3d

# Atau via chocolatey
choco install k3d

# Buat cluster K3s
k3d cluster create todo-cluster \
  --servers 1 \
  --agents 2 \
  --port "30080-30100:30080-30100@server:0" \
  --k3s-arg "--disable=traefik@server:0"

# Verifikasi
kubectl cluster-info
kubectl get nodes

```

2.2 Konfigurasi Lingkungan A: Docker Compose (Statik)

Buka powershell dan jalankan perintah berikut:

```

# Buat folder environment A

```

```
mkdir C:\experiment\env-a
cd C:\experiment\env-a

# Copy aplikasi
xcopy C:\experiment\todo-research\to-do-list-app . /E

# Build dan jalankan
docker-compose build
docker-compose up -d

# Verifikasi semua container running
docker-compose ps

# Test endpoint
curl http://localhost:3000/health
curl http://localhost:3001/health
```

```
curl http://localhost:3002/health
```

2.3 Konfigurasi Lingkungan B: K3s dengan HPA

Buka powershell dan jalankan perintah berikut:

```
# Setup K3d context
k3d kubeconfig merge todo-cluster -d C:\experiment\env-b\kubeconfig.yaml
$env:KUBECONFIG = "C:\experiment\env-b\kubeconfig.yaml"

# Build images ke dalam K3d cluster
k3d image import auth-service:latest -c todo-cluster
k3d image import todo-service:latest -c todo-cluster
k3d image import notification-service:latest -c todo-cluster
k3d image import frontend:latest -c todo-cluster

# Apply manifests
kubectl apply -f k3s-manifests/

# Tunggu semua pods running
kubectl wait --for=condition=ready pod --all -n todo-research --timeout=300s

# Verifikasi HPA
kubectl get hpa -n todo-research

# Setup port forwarding untuk akses
kubectl port-forward -n todo-research service/frontend 8081:80 &
```

FASE 3: PENGUJIAN DAN ANALISIS

3.1 Simulasi Load Testing menggunakan Apache JMeter

3.1.1 Install JMeter di Windows

Buka powershell dan jalankan perintah berikut:

```
# Download JMeter
cd C:\experiment\tools
Invoke-WebRequest -Uri "https://dlcdn.apache.org/jmeter/binaries/apache-jmeter-5.6.3.zip" -OutFile
"jmeter.zip"

# Extract
Expand-Archive -Path jmeter.zip -DestinationPath .
```

```
# Tambahkan ke PATH
$env:Path += ";C:\experiment\tools\apache-jmeter-5.6.3\bin"
```

3.1.2 Membuat Test Plan JMeter

📄 todo-load-test.jmx (Simpan file ini)

```
<?xml version="1.0" encoding="UTF-8"?>
<jmeterTestPlan version="1.2" properties="5.0" jmeter="5.6.3">
  <hashTree>
    <TestPlan guiclass="TestPlanGui" testclass="TestPlan" testname="To-Do List Load Test">
      <elementProp name="TestPlan.user_defined_variables" elementType="Arguments">
        <collectionProp name="Arguments.arguments">
          <elementProp name="BASE_URL_A" elementType="Argument">
            <stringProp name="Argument.name">BASE_URL_A</stringProp>
            <stringProp name="Argument.value">http://localhost:8080</stringProp>
          </elementProp>
          <elementProp name="BASE_URL_B" elementType="Argument">
            <stringProp name="Argument.name">BASE_URL_B</stringProp>
            <stringProp name="Argument.value">http://localhost:8081</stringProp>
          </elementProp>
        </collectionProp>
      </elementProp>
    </TestPlan>
  </hashTree>
  <!-- Thread Group untuk Ramp Up Test -->
  <ThreadGroup guiclass="ThreadGroupGui" testclass="ThreadGroup" testname="Ramp Up Test - 100 Users">
    <intProp name="ThreadGroup.num_threads">100</intProp>
    <intProp name="ThreadGroup.ramp_time">60</intProp>
    <longProp name="ThreadGroup.duration">300</longProp>
    <boolProp name="ThreadGroup.scheduler">true</boolProp>
  </ThreadGroup>
  <hashTree>
    <!-- CSV Data Set untuk user credentials -->
    <CSVDataSet guiclass="TestBeanGUI" testclass="CSVDataSet" testname="User Credentials">
      <stringProp name="filename">users.csv</stringProp>
      <stringProp name="variableNames">username,password</stringProp>
      <boolProp name="shareMode">shareMode.all</boolProp>
    </CSVDataSet>

    <!-- HTTP Request Defaults -->
    <ConfigTestElement guiclass="HttpDefaultsGui" testclass="ConfigTestElement" testname="HTTP Request Defaults">
      <elementProp name="HTTPSampler.Arguments" elementType="Arguments">
        <collectionProp name="Arguments.arguments">
          <elementProp name="Content-Type" elementType="HTTPArgument">
            <stringProp name="Argument.value">application/json</stringProp>
            <stringProp name="Argument.metadata">=</stringProp>
          </elementProp>
        </collectionProp>
      </elementProp>
      <boolProp name="HTTPSampler.image_parser">false</boolProp>
    </ConfigTestElement>

    <!-- HTTP Header Manager -->
    <HeaderManager guiclass="HeaderPanel" testclass="HeaderManager" testname="HTTP Header Manager">
      <collectionProp name="HeaderManager.headers">
        <elementProp name="" elementType="Header">
          <stringProp name="Header.name">Content-Type</stringProp>

```

```

        <stringProp name="Header.value">application/json</stringProp>
    </elementProp>
</collectionProp>
</HeaderManager>

<!-- Login Request -->
<HTTPSamplerProxy          guiclass="HttpTestSampleGui"          testclass="HTTPSamplerProxy"
testname="Login">
    <stringProp name="HTTPSampler.domain">${__P(BASE_URL)}</stringProp>
    <stringProp name="HTTPSampler.path">/api/auth/login</stringProp>
    <stringProp name="HTTPSampler.method">POST</stringProp>
    <elementProp name="HTTPSampler.Arguments" elementType="Arguments">
        <collectionProp name="Arguments.arguments">
            <elementProp name="" elementType="HTTPArgument">
                <stringProp
name="Argument.value">{"username":"${username}","password":"${password}"}</stringProp>
                <stringProp name="Argument.metadata">=</stringProp>
            </elementProp>
        </collectionProp>
    </elementProp>
    <boolProp name="HTTPSampler.postBodyRaw">true</boolProp>
</HTTPSamplerProxy>
<hashTree>
    <!-- JSON Extractor untuk token -->
    <JSONPostProcessor      guiclass="JSONPostProcessorGui"      testclass="JSONPostProcessor"
testname="Extract Token">
        <stringProp name="JSONPostProcessor.referenceNames">authToken</stringProp>
        <stringProp name="JSONPostProcessor.jsonPathExprs">$.token</stringProp>
    </JSONPostProcessor>
</hashTree>

    <!-- Get Todos (70% traffic) -->
    <HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy" testname="Get All
Todos">
        <stringProp name="HTTPSampler.domain">${__P(BASE_URL)}</stringProp>
        <stringProp name="HTTPSampler.path">/api/todos</stringProp>
        <stringProp name="HTTPSampler.method">GET</stringProp>
        <elementProp name="HTTPSampler.Arguments" elementType="Arguments">
            <collectionProp name="Arguments.arguments">
                <elementProp name="Authorization" elementType="HTTPArgument">
                    <stringProp name="Argument.value">Bearer ${authToken}</stringProp>
                    <stringProp name="Argument.metadata">=</stringProp>
                </elementProp>
            </collectionProp>
        </elementProp>
    </HTTPSamplerProxy>

    <!-- Create Todo (20% traffic) -->
    <HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy" testname="Create
Todos">
        <stringProp name="HTTPSampler.domain">${__P(BASE_URL)}</stringProp>
        <stringProp name="HTTPSampler.path">/api/todos</stringProp>
        <stringProp name="HTTPSampler.method">POST</stringProp>
        <elementProp name="HTTPSampler.Arguments" elementType="Arguments">
            <collectionProp name="Arguments.arguments">
                <elementProp name="Authorization" elementType="HTTPArgument">
                    <stringProp name="Argument.value">Bearer ${authToken}</stringProp>
                    <stringProp name="Argument.metadata">=</stringProp>
                </elementProp>
                <elementProp name="" elementType="HTTPArgument">
                    <stringProp
test task">{"title":"Task  ${__time()}", "description":"Load
                    <stringProp name="Argument.metadata">=</stringProp>
                </elementProp>
            </collectionProp>
        </elementProp>
    </HTTPSamplerProxy>

```

```

        </collectionProp>
        </elementProp>
        <boolProp name="HTTPSampler.postBodyRaw">true</boolProp>
    </HTTPSamplerProxy>

    <!-- Response Assertions -->
    <AssertionResultCollector                                guiclass="AssertionVisualizer"
testclass="AssertionResultCollector" testname="View Results Tree"/>

    <!-- Summary Report -->
    <SummaryReport    guiclass="SummaryReportGui"    testclass="SummaryReport"    testname="Summary
Report"/>
    </hashTree>
    </hashTree>
    </hashTree>
</jmeterTestPlan>

```

3.1.3 File Data Users (users.csv)

Buat file users.csv di C:\experiment\data\:

```

username,password
testuser1,testpass123
testuser2,testpass123
testuser3,testpass123
testuser4,testpass123
testuser5,testpass123

```

3.1.4 Menjalankan Load Test

Test Scenario 1: Baseline (10 users)

Buka powershell dan jalankan perintah berikut:

```

# Test Environment A
jmeter -n -t todo-load-test.jmx -l results_env_a_baseline.csv -JBASE_URL=http://localhost:8080 -
Jusers=10 -Jduration=120

# Test Environment B
jmeter -n -t todo-load-test.jmx -l results_env_b_baseline.csv -JBASE_URL=http://localhost:8081 -
Jusers=10 -Jduration=120

```

Test Scenario 2: Ramp Up Test (10 → 200 users)

Buka powershell dan jalankan perintah berikut:

```

# Script untuk ramp up test
for ($users = 10; $users -le 200; $users += 20) {
    Write-Host "Testing with $users users..."

    # Environment A
    jmeter -n -t todo-load-test.jmx -l "results_a_${users}.csv" `
        -JBASE_URL=http://localhost:8080 -Jusers=$users -Jduration=60

    # Environment B
    jmeter -n -t todo-load-test.jmx -l "results_b_${users}.csv" `
        -JBASE_URL=http://localhost:8081 -Jusers=$users -Jduration=60

    Start-Sleep -Seconds 30 # Cool down
}

```

Test Scenario 3: Stress Test (Spike)

Buka powershell dan jalankan perintah berikut:

```
# Spike: tiba-tiba naik ke 500 users
jmeter -n -t spike_test.jmx -l results_a_spike.csv -JBASE_URL=http://localhost:8080 -Jusers=500
```

3.2 Ekstraksi Metrik via Prometheus & Grafana (Windows Native)

3.2.1 Install Prometheus Windows Exporter

Buka powershell dan jalankan perintah berikut:

```
# Download Windows Exporter
cd C:\experiment\tools
Invoke-WebRequest -Uri "https://github.com/prometheus-community/windows_exporter/releases/download/v0.25.1/windows_exporter-0.25.1-amd64.msi" -OutFile "windows_exporter.msi"

# Install
msiexec /i windows_exporter.msi ENABLED_COLLECTORS="cpu,memory,net,process,container"

# Verifikasi
netstat -ano | findstr :9182
```

3.2.2 Install Prometheus

Buka powershell dan jalankan perintah berikut:

```
# Download Prometheus
Invoke-WebRequest -Uri "https://github.com/prometheus/prometheus/releases/download/v2.53.0/prometheus-2.53.0.windows-amd64.zip" -OutFile "prometheus.zip"
Expand-Archive -Path prometheus.zip -DestinationPath .

# Buat config file
@"
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'windows'
    static_configs:
      - targets: ['localhost:9182']

  - job_name: 'docker'
    static_configs:
      - targets: ['localhost:9323'] # Docker metrics

  - job_name: 'kubernetes'
    kubernetes_sd_configs:
      - api_server: 'https://localhost:6443'
        tls_config:
          insecure_skip_verify: true
"@ | Out-File -FilePath prometheus.yml

# Jalankan Prometheus
```

```
.\prometheus.exe --config.file=prometheus.yml --web.listen-address=:9090
```

3.2.3 Install Grafana

Buka powershell dan jalankan perintah berikut:

```
# Install Grafana via winget
winget install GrafanaLabs.Grafana

# Start Grafana service
net start Grafana

# Buka Grafana di browser
Start-Process "http://localhost:3000"
# Login: admin / admin
```

3.2.4 Dashboard Monitoring

Import dashboard berikut (ID: 193, 1860, 179):

1. Docker Monitoring Dashboard
2. Kubernetes Cluster Monitoring
3. JMeter Dashboard

3.2.5 Script Ekstraksi Metrik Otomatis

📄 collect_metrics.ps1

```
param(
    [string]$Environment,
    [string]$TestName,
    [int]$DurationSeconds = 300
)

$outputDir = "C:\experiment\results\$Environment"
New-Item -ItemType Directory -Force -Path $outputDir

Write-Host "Collecting metrics for $Environment - $TestName"

# Collect Docker stats (Environment A only)
if ($Environment -eq "env_a") {
    $dockerStats = docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
    $dockerStats | Out-File "$outputDir\docker_stats_$TestName.txt"
}

# Collect Kubernetes metrics (Environment B only)
if ($Environment -eq "env_b") {
    kubectl top pods -n todo-research | Out-File "$outputDir\k8s_pods_stats_$TestName.txt"
    kubectl get hpa -n todo-research | Out-File "$outputDir\hpa_status_$TestName.txt"
}

# Collect Windows resource usage
$cpu = Get-Counter "\Processor(_Total)\% Processor Time"
$memory = Get-Counter "\Memory\Available MBytes"
$disk = Get-Counter "\PhysicalDisk(_Total)\% Disk Time"

# Collect container metrics via Prometheus API
```

```

$promQuery = "http://localhost:9090/api/v1/query?query=container_cpu_usage_seconds_total"
$metrics = Invoke-RestMethod -Uri $promQuery

# Simpan ke CSV
$timestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$metricData = [PSCustomObject]@{
    Timestamp = $timestamp
    Environment = $Environment
    TestName = $TestName
    CPU_Total = $cpu.CounterSamples.CookedValue
    Memory_Available_MB = $memory.CounterSamples.CookedValue
    Disk_Usage = $disk.CounterSamples.CookedValue
    Container_Count = (docker ps -q | Measure-Object).Count
}

$metricData | Export-Csv -Path "$outputDir\metrics_$TestName.csv" -Append

Write-Host "Metrics collected successfully"

```

3.2.6 Template Metrik yang Dikumpulkan Form Data Collection Sheet (Excel)

Test Run	Environment	User Load	Avg Response (ms)	P95 Response	Error Rate	CPU %	Memory MB	Pods/Containers	HPA Status
1	Docker Compose	10							N/A
1	Docker Compose	50							N/A
1	Docker Compose	100							N/A
1	Docker Compose	200							N/A
2	Kubernetes	10							1 pod
2	Kubernetes	50							1-2 pods
2	Kubernetes	100							2-4 pods
2	Kubernetes	200							4-6 pods

3.3 Analisis Komparatif Skalabilitas dan Efisiensi Sumber Daya

3.3.1 Metrik Analisis

A. Skalabilitas

1. Throughput maksimum (requests/second)
2. Waktu respons pada berbagai tingkat beban
3. Titik jenuh (saturation point) dimana error rate > 1%
4. Kemampuan horizontal scaling (K8s) vs vertical (Compose)

B. Efisiensi Resource

1. Resource Utilization Score (RUS) = (Throughput) / (CPU_used + Memory_used/1024)
2. Overhead orchestration = Resource K8s - Resource Compose pada idle load
3. Cost Efficiency Index (CEI) = Throughput / (\$cost_per_hour)

C. Reliability

- 1. Error rate distribution
- 2. Recovery time setelah spike
- 3. Fault tolerance (matikan 1 pod/service)

3.3.2 Template Analysis

analysis_report.md

Hasil Analisis Komparatif

1. Perbandingan Skalabilitas

Throughput vs User Load

User Load	Docker Compose (req/s)	Kubernetes (req/s)	Improvement
10			
50			
100			
200			
500			

Response Time (P95)

User Load	Docker Compose (ms)	Kubernetes (ms)	SLA Met?
10			
50			
100			
200			

2. Perbandingan Efisiensi Resource

Idle State (0 users)

Resource	Docker Compose	Kubernetes	Overhead
CPU			
Memory			
Disk I/O			

Peak Load (200 users)

Resource	Docker Compose	Kubernetes	Efficiency
REI Score			
CPU per req			
Memory per req			

3. Scaling Behavior Analysis (Kubernetes Only)

Time (min)	User Load	Active Pods	CPU Avg (%)	Scaling Event
0-2	10-50			
2-4	50-150			Scale up to 3
4-6	150-300			Scale up to 5
6-8	300-100			Scale down to 3
8-10	100-10			Scale down to 1

Scaling latency: __ seconds from load increase to pod creation
Cool down time: __ seconds from load decrease to pod termination

4. Kesimpulan Utama

Kelebihan Docker Compose:

1.

- 2.
- 3.

Kelebihan Kubernetes:

- 1.
- 2.
- 3.

Rekomendasi Deployment:

- Gunakan Docker Compose untuk:
- Gunakan Kubernetes untuk:

3.4 Penarikan Kesimpulan dan Saran

Template Kesimpulan

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil pengujian load testing pada aplikasi **To-Do List** microservices dengan **100-200** user concurrent, diperoleh kesimpulan sebagai berikut:

Skalabilitas

1. Docker Compose (Manual Scaling):

- Mampu menangani maksimal ____ req/s dengan latency p95 ____ ms
- Saturasi terjadi pada ____ users dengan error rate ____%
- Scaling memerlukan intervensi manual selama ____ menit

2. Kubernetes dengan HPA (Auto-scaling):

- Mampu menangani maksimal ____ req/s (____% lebih tinggi dari Compose)
- Auto-scaling bereaksi dalam ____ detik saat CPU melebihi 60%
- Stabil pada beban ____ users dengan ____ pods aktif

Efisiensi Resource

1. Pada beban rendah (0-50 users):

- Docker Compose ____% lebih efisien (REI score: ____ vs ____)
- Overhead Kubernetes sebesar ____ CPU core dan ____ MB memory

2. Pada beban tinggi (>150 users):

- Kubernetes ____% lebih efisien karena distribusi beban
- Resource utilization lebih merata (std dev ____% vs ____%)

3. Trade-off signifikan:

- Compose: setup sederhana, overhead kecil, scaling manual
- Kubernetes: overhead besar di idle, auto-scaling powerful

Hipotesis Terbukti

- [] Hipotesis 1: Kubernetes memiliki skalabilitas lebih tinggi
- [] Hipotesis 2: Docker Compose lebih efisien pada beban rendah
- [] Hipotesis 3: HPA berhasil mempertahankan latency <500 ms
- [] Hipotesis 4: Overhead orchestration signifikan (____% resource)

5.2 Saran untuk Pengembangan

Untuk Aplikasi Serupa (To-Do List):

1. Gunakan Docker Compose jika:

- User concurrent < 50
- Budget resource terbatas (RAM < 4GB)
- Development/testing environment
- Traffic pattern stabil

2. Gunakan Kubernetes jika:

- User concurrent > 100
- Ada lonjakan traffic tidak terduga
- Memerlukan high availability (99.9% uptime)
- Tim memiliki expertise K8s

3. Optimasi yang disarankan:

- Set HPA threshold lebih rendah (40%) untuk reaksi lebih cepat
- Tambahkan cluster autoscaler untuk node scaling
- Implementasi cache (Redis) untuk reduce database load
- Gunakan connection pooling untuk database

LAMPIRAN: SCRIPT OTOMATISASI EKSPERIMEN

A. Script Runner Lengkap (Windows PowerShell)

Buatlah file *run_full_experiment.ps1* dan isikan kode program berikut:

```
param(
    [ValidateSet("env_a","env_b","both")]
    [string]$Target = "both"
)

Write-Host "===== " -ForegroundColor Cyan
Write-Host "TO-DO LIST MICROSERVICES EXPERIMENT" -ForegroundColor Cyan
Write-Host "Platform: Windows 11" -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan

# Setup directories
$experimentRoot = "C:\experiment"
$resultsDir = "$experimentRoot\results\$(Get-Date -Format 'yyyyMMdd_HH:mm:ss')"
New-Item -ItemType Directory -Force -Path $resultsDir

# Function to test environment
function Test-Environment {
    param(
        [string]$Name,
        [string]$Url,
        [int]$StartUsers = 10,
        [int]$EndUsers = 200,
        [int]$Step = 20
    )

    Write-Host "`n[TESTING] Environment: $Name" -ForegroundColor Yellow
    Write-Host "URL: $Url" -ForegroundColor Gray

    $envResultsDir = "$resultsDir\$Name"
    New-Item -ItemType Directory -Force -Path $envResultsDir

    # Baseline test
    Write-Host "`n>>> Running baseline test (10 users)..." -ForegroundColor Green
    jmeter -n -t todo-load-test.jmx -l "$envResultsDir\baseline.csv" `
        -JBASE_URL=$Url -Jusers=10 -Jduration=120

    # Ramp up tests
    for ($users = $StartUsers; $users -le $EndUsers; $users += $Step) {
        Write-Host "`n>>> Testing with $users users..." -ForegroundColor Green

        # Start monitoring
        Start-Job -Name "metrics_$Name`_$users" -ScriptBlock {
            param($env, $users, $outDir)
            C:\experiment\scripts\collect_metrics.ps1 -Environment $env -TestName "load_$users"
        } -ArgumentList $Name, $users, $envResultsDir
    }
}
```

```

# Run JMeter test
jmeter -n -t todo-load-test.jmx -l "$envResultsDir\load_${users}.csv" `
    -JBASE_URL=$Url -Jusers=$users -Jduration=180

# Stop monitoring
Stop-Job -Name "metrics_${Name}_${users}"

# Cool down
Write-Host "Cooling down for 30 seconds..." -ForegroundColor Gray
Start-Sleep -Seconds 30
}

# Stress test (spike)
Write-Host "`n>>> Running stress test (spike to 500 users)..." -ForegroundColor Red
jmeter -n -t spike_test.jmx -l "$envResultsDir\spike.csv" `
    -JBASE_URL=$Url -Jusers=500 -Jduration=60

Write-Host "[COMPLETED] Environment $Name" -ForegroundColor Green
}

# Run tests
if ($Target -in @("env_a","both")) {
    Write-Host "`n=== Starting Environment A (Docker Compose) ===" -ForegroundColor Magenta
    docker-compose -f $experimentRoot\env-a\docker-compose.yml up -d
    Start-Sleep -Seconds 30
    Test-Environment -Name "env_a" -Url "http://localhost:8080"
    docker-compose -f $experimentRoot\env-a\docker-compose.yml down
}

if ($Target -in @("env_b","both")) {
    Write-Host "`n=== Starting Environment B (K3s Kubernetes) ===" -ForegroundColor Magenta
    Test-Environment -Name "env_b" -Url "http://localhost:8081"
}

# Generate comparison report
Write-Host "`n=== Generating Analysis Report ===" -ForegroundColor Cyan
& "C:\experiment\scripts\generate_report.ps1" -ResultsDir $resultsDir

Write-Host "`n===== " -ForegroundColor Cyan
Write-Host "EXPERIMENT COMPLETED!" -ForegroundColor Green
Write-Host "Results saved to: $resultsDir" -ForegroundColor Green
Write-Host "===== " -ForegroundColor Cyan

```

1. Buka PowerShell sebagai Administrator

- Klik **Start Menu** Windows.
- Ketik **PowerShell**.
- Klik kanan pada **Windows PowerShell** (atau **Terminal**).
- Pilih **Run as administrator**.

2. Atur Kebijakan Eksekusi (Execution Policy)

Secara default, Windows 11 memblokir eksekusi skrip luar demi keamanan. Izinkan eksekusi skrip untuk sesi ini saja dengan mengetik perintah berikut lalu tekan **Enter**:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

Use code with caution.

3. Masuk ke Folder Skrip

Gunakan perintah `cd` untuk berpindah ke direktori tempat Anda menyimpan file skrip tersebut. Contoh jika file ada di direktori `C:\experiment`:

powershell

```
cd C:\experiment
```

Use code with caution.

4. Jalankan Skrip

Anda memiliki tiga opsi cara menjalankan skrip ini tergantung kebutuhan pengujian Anda:

Opsi A: Menguji Semua Lingkungan (Default)

Skrip akan menguji `env_a` (Docker Compose) sekaligus `env_b` (K3s).

```
.\run_full_experiment.ps1
```

Use code with caution.

Opsi B: Menguji Environment A Saja

Hanya menguji microservices yang berjalan di Docker Compose.

```
.\run_full_experiment.ps1 -Target "env_a"
```

Use code with caution.

Opsi C: Menguji Environment B Saja

Hanya menguji microservices yang berjalan di K3s Kubernetes.

```
.\run_full_experiment.ps1 -Target "env_b"
```

B. Troubleshooting Windows-Specific Issues

Masalah	Solusi
Docker Desktop WSL error	<code>wsl --shutdown</code> lalu restart Docker
Port already in use	<code>netstat -ano findstr :3000</code> lalu <code>taskkill /PID xxx /F</code>
K3d cluster gagal	<code>k3d cluster delete</code> lalu <code>k3d cluster create --config</code>
JMeter out of memory	Set HEAP: <code>set HEAP=-Xms2g -Xmx4g</code> di <code>jmeter.bat</code>
Prometheus can't scrape	Cek firewall: <code>New-NetFirewallRule -DisplayName "Prometheus" -Direction Inbound -Protocol TCP -LocalPort 9090 -Action Allow</code>